Selectively Permeable Subagent Contexts with Progressive Retrieval Attention

PRA Research Program

September 2026

Abstract

Subagents isolate execution, but their context boundary is usually coarse. A child either receives a copied parent transcript or reacquires information, and a parent commonly receives a lossy child summary. We extend Progressive Retrieval Attention (PRA) from one typed-record stream to a lineage graph with *selectively permeable context boundaries*. The implementation assigns records to agents, persists start/stop/resume events, exposes selected or routable ancestor and completed-descendant records, and validates exact reuse through four generic effects and normalized resource identities. Concrete tool semantics remain in the harness; the runtime remains tool agnostic.

We separate three gains that are often conflated: avoided tool execution, payload reuse, and native K/V reuse. The initial Subagent Context Reuse Benchmark (SCRB) is a five-seed, injected-cost systems simulation with direct runtime contract checks, not a live-engine timing result. At 8K shared tokens and eight children, memoization removes eight repeated tool calls but retains 73,728 prefill tokens. Compatible native reuse reduces prefill to 8,192 tokens and the declared session cost from 1,924.6 to 313.1 ms, a simulated 6.15× speedup. Under a relevant write, resource-level invalidation preserves all unmodified-resource hits with zero stale reads; domain-wide invalidation loses them, while an unsafe control produces 16 stale reads at fan-out 16. In a completed-child evidence task, selective descendant routing attains the full-transcript upper bound with 384 rather than 16,384 parent-context tokens. An opportunity audit of 12 existing single-agent coding traces finds eight exact repeated reads/searches among 40 such calls, but does not establish a natural cross-agent hit rate. Live native-engine and natural subagent results therefore remain open gates.

1 Introduction

Subagents have become a common agent-harness primitive. A parent model issues a tool-like delegation command, the harness creates a child model loop, and the child operates in a separate context until it returns a result, summary, artifact, or status. This design provides useful context isolation and enables parallel or specialist work, but the boundary is usually coarse. The child either begins with little parent state or receives a copied/summarized subset, and the parent generally sees only a compact return value rather than an addressable representation of the child's detailed execution history.

This paper studies a complementary question: *what if subagent context boundaries remain execution-isolated but become selectively permeable at the record and native-attention-state levels?*

The work builds on earlier PRA papers on retrieval within attention, progressive materialization, persistent and typed records, tool/skill integration, structured context, and task-aware compaction. In particular, Paper 8 establishes a single-session agent harness where tool calls, tool results, tasks, artifacts, and compacted representations are explicit PRA records. Paper 9 extends this model across

multiple agents by adding agent identity and lineage, a context-stream tree or DAG, cross-stream visibility, and validity-aware reuse.

The design deliberately separates three layers:

1. the **agent harness**, which understands concrete tool semantics and ordinary subagent execution;
2. the **PRA runtime**, which may be remote and therefore operates only on generic records, resource/effect metadata, visibility, validity, lineage, and policy;
3. the **PRA-enabled model engine**, which stores and materializes native K/V and performs model-specific routing and context integration.

The resulting abstraction is a graph of context streams rather than a collection of isolated transcript blobs.

Contributions. This paper contributes:

1. a generic representation of agent lineage and subagent lifecycle in PRA records;
2. a tool-agnostic declarative effect/resource model for determining when prior records remain valid;
3. ancestor and descendant record visibility policies that do not require copying entire contexts;
4. cross-agent tool-result and payload reuse;
5. native K/V reuse across agent streams when model and engine constraints permit;
6. controlled consistency semantics for sequential, parallel, shared-workspace, and externally mutable resources;
7. a benchmark suite designed specifically to quantify when cross-agent reuse produces a performance benefit;
8. a controlled benchmark and an instrumentation-only audit that make the current evidence boundary explicit.

2 Background and Position in the PRA Series

2.1 Progressive Retrieval Attention

PRA treats logical context as larger than the physically materialized model context. External records can remain addressable and be selectively routed and materialized into attention as needed [3, 4]. Earlier papers developed semantic, lexical, hierarchical, iterative, adaptive, and graph-aware selection mechanisms, as well as model/engine integrations capable of retaining native per-layer K/V state for external records.

2.2 From persistent records to agent context

The Papers 5–7 line develops persistent and typed resources, runtime integration, adaptive context control, and structured records. Paper 8 moves these ideas into an agent harness with task records and explicit tool-call/tool-result records. Tool results can be compacted in active context while preserving richer native/detail representations externally.

Paper 9 starts from this property:

> Compaction need not mean deletion. A compact active representation may coexist with a richer addressable representation and, where supported, persisted native K/V.

This makes subagent reuse possible even after the parent has compacted its own tool history.

2.3 Relationship to Paper 4.5 and the agent SDK

Paper 4.5 establishes practical deployment modes and SDK boundaries among agent-side behavior, PRA gateways/runtimes, and engine integration. Paper 9 should preserve these boundaries. A PRA runtime may execute out-of-process or remotely and must not depend on Python objects, local filesystem assumptions, or a particular harness implementation.

3 Conventional Subagent Mechanics

At the harness level, the initial Paper 9 implementation should intentionally remain conventional:

```
Parent LLM
  -> spawn_subagent(task, options)
Harness
  -> allocate child agent/session
  -> select model/tools/workspace/context policy
Child agent loop
  -> LLM -> tools -> observations -> ...
Harness
  -> return result/summary/artifacts
Parent LLM
```

The paper does not claim novelty in this mechanism. Novelty begins after child creation: how the parent and child context streams are represented, what records are mutually visible, and whether already acquired/encoded information can be safely reused.

4 System Architecture

4.1 Agent harness responsibilities

The harness is responsible for:

- spawning, resuming, stopping, and inspecting subagents;
- assigning agent and task identities;
- selecting child models, tools, permissions, and workspace semantics;
- understanding concrete tool behavior;

- mapping concrete tools to generic effect/resource descriptors;
- emitting typed lifecycle and tool records;
- enforcing safety and execution permissions;
- applying user-configured subagent semantics.

The harness should be able to expose a capability matrix rather than requiring all semantics in the first implementation.

4.2 PRA runtime responsibilities

The PRA runtime should understand only generic concepts:

- session, agent, task, and record identities;
- parent/child lineage and context-stream topology;
- typed records and compact/detail representations;
- record visibility and routing eligibility;
- resource domains and resource identities;
- generic effects and dependency sets;
- validity and reuse policies;
- causal/logical ordering;
- materialization state and engine capability descriptors.

It should not contain special cases for tools such as `file_read`, `git_diff`, SQL, or HTTP.

4.3 PRA engine responsibilities

The engine integration handles model-near state and execution:

- persisted native K/V;
- model-specific tokenization and positional handling;
- routing and per-layer materialization;
- cross-agent K/V reuse when compatible;
- cache residency and eviction;
- bounded physical context.

5 Agent and Record Topology

Each PRA record gains an `agent_uuid` in addition to existing session and record metadata.

```
RecordMetadata
  record_uuid
  session_uuid
  agent_uuid
  task_uuid?
  created_at
  logical_clock?
  type
  representation
```

Subagent creation is recorded as a lifecycle event, usually caused by an ordinary tool call in the parent stream:

```
AgentStartRecord
  agent_uuid
  session_uuid
  parent_agent_uuid
  parent_task_uuid?
  spawn_call_record_uuid?
  model_descriptor
  workspace_id?
  tool_profile_id?
  context_policy
```

A corresponding `AgentStopRecord` captures status and typed result, summary, and artifact references. Completed child branches can remain externally addressable even if no longer active.

The resulting context topology may be a tree under simple delegation, or a DAG if later semantics permit shared tasks, joins, or explicitly shared context branches.

6 Configurable Subagent Semantics

Table 1 lists the main dimensions exposed by the harness SDK.

Dimension	Candidate semantics
Child context	empty; selected; parent snapshot; logical fork
Child model	same; cheaper; specialist; explicit
Tools	inherited; restricted; specialist; explicit
State	ephemeral; persistent; resumable
Execution	synchronous; asynchronous
Scheduling	sequential; parallel; DAG
Return	final answer; summary; artifacts; typed records
Communication	child-to-parent; bidirectional; peer-to-peer
Workspace	shared; isolated; worktree/branch; explicit
Lifetime	one-shot; resumable
Recursion	prohibited; bounded; unrestricted
Ancestor visibility	none; selected; routable; inherited
Descendant visibility	none; completed-only; routable

Table 1: Subagent semantics exposed by the harness SDK. Paper 9 need not implement all modes initially.

7 Generic Effect and Resource Descriptors

Cross-agent reuse is unsafe unless the system can determine whether a prior result remains valid. PRA should not learn tool-specific semantics. Instead, harness adapters map tools into a deliberately small generic effect system:

$$\texttt{EffectType} \in \{\texttt{PURE}, \texttt{READ}, \texttt{WRITE}, \texttt{UNKNOWN}\}.$$

A read tool may declare a resource domain and a key derived from its arguments:

```
name: file_read

effect: READ
resource_domain: os.FILE
resource_key: normalize(args.path)
reuse:
  enabled: true
  max_age: 300s
  validation: stat
```

A corresponding write declares mutation and invalidation:

```
name: file_write

effect: WRITE
resource_domain: os.FILE
resource_key: normalize(args.path)
invalidates: same_resource
```

The runtime sees only normalized fields. For example, the domain is `os.FILE`, and the key is `/repo/src/parser.py`.

7.1 Dependency sets

Some results depend on multiple or coarse-grained resources. A directory grep can declare dependencies on a file tree or a resolved set of matching files. This turns validity tracking into a generic dependency/invalidation problem rather than a filesystem-specific optimization.

8 Selective Context Permeability

A child does not need a physical copy of parent context. Instead, its logical context can be represented as

$$C_{child} = C_{local} \cup C_{inherited} \cup C_{ancestor-visible},$$

where the third set remains external until routed/materialized.

The most important mode is `ROUTABLE`: ancestor records remain candidates for PRA selection but are not physically inserted into the child's prompt or K/V until needed.

The reverse direction is also useful. A parent may route into completed descendant records when a later synthesis question requires detailed evidence unavailable in the child's returned summary.

9 Three Levels of Cross-Agent Reuse

9.1 Tool execution reuse

If a child requests an operation whose valid result already exists in an ancestor-visible stream, the system can avoid executing the external tool again.

9.2 Record payload reuse

The prior result record can be referenced directly instead of copied and re-serialized into the child stream.

9.3 Native K/V reuse

If the PRA engine retains compatible native K/V for that record, it can materialize the existing state into the child context without repeated model prefill. This is the most PRA-specific benefit and distinguishes the proposal from ordinary harness-level memoization.

10 Consistency and Validity

Parallel agents and external processes make cache validity nontrivial. We therefore define explicit consistency modes.

ANCESTRY. Only mutations in the child-visible ancestor causal chain invalidate a prior result. This is appropriate for immutable snapshots or isolated worktrees.

SESSION_TREE. Any causally prior mutation to the relevant resource in the shared session tree invalidates. This assumes the harness controls the shared workspace.

EXTERNAL. Resources may change outside PRA. Reuse requires TTL, file metadata, content hashes, ETags, version tokens, or another application-provided validator; otherwise reuse is disabled.

Conservative invalidation is the default. Unknown effects do not qualify for reuse.

11 Why Compaction Matters

Paper 8 may compact a large tool result into a smaller active representation while retaining a richer addressable representation and possibly persisted native K/V. Therefore, compaction does not remove the opportunity for a later child to reuse the original information.

A conventional compacting agent may need to execute the tool again because the detailed output is no longer present in active context. PRA can instead route to the original record and rematerialize its payload or native K/V into the child context, provided validity checks succeed.

12 Benchmark: Subagent Context Reuse Benchmark (SCRB)

A generic coding benchmark may contain too few repeated parent-child reads to measure this mechanism reliably. We therefore propose SCRB, a controlled suite in which reuse opportunities are intentional and parameterized, followed by naturalistic validation on existing coding-agent tasks.

12.1 Family A: Shared repository orientation

The parent reads common repository instructions, architecture documents, build metadata, core interfaces, and test conventions before spawning multiple independent children. Children solve separate subtasks that naturally require the same shared material.

Primary variables:

- number of children: 2, 4, 8, 16;
- shared context size;
- probability that a child needs each shared record;
- whether the common material is still active, compacted, or external-only.

12.2 Family B: Shared definition fan-out

Repositories contain a large protocol schema, AST definition, API interface, generated type file, or configuration model needed by many independent children. The parent reads it once. Children perform separate fixes or analyses that all depend on it.

Sweep shared definition sizes such as 2K, 8K, 32K, and 64K tokens to expose the transition where native K/V reuse becomes valuable.

12.3 Family C: Repeated grep/search

The parent performs repository-wide searches for symbol usages, route registrations, interface implementations, error codes, or table references. Children later issue equivalent searches. Variants include exact repeated queries, semantically equivalent but syntactically different queries, and writes that should or should not invalidate the cached search result.

12.4 Family D: Unrelated mutation

The parent reads resources A and B. One child writes A while another later reads B. Correct selective invalidation should preserve reuse of B, unlike coarse domain-wide invalidation.

12.5 Family E: Relevant mutation

The parent reads A, a child writes A, and a later child reads A. Reuse must be rejected under shared-workspace semantics. The primary safety metric is stale-read rate, which should be zero for supported consistency modes.

12.6 Family F: Isolated worktrees

Each child operates in a separate Git worktree or immutable snapshot. Sibling mutations should not invalidate ancestor reads in another worktree when resource identities and workspace IDs establish isolation. This family studies how workspace semantics alter the safe reuse envelope.

12.7 Family G: Completed-child evidence recovery

Several children investigate different hypotheses and return compact summaries. The parent then receives a synthesis question requiring exact evidence from one completed child, such as a test log, code span, or interface definition. Compare summary-only, full-transcript copying, transcript replay, and PRA descendant routing.

12.8 Family H: Hierarchical delegation

Root agents spawn children that spawn grandchildren. Common instructions or interfaces originate in the root. This family measures whether deeper descendants can reuse valid root records without full context copying as delegation depth increases.

12.9 Family I: Expensive remote tools

Controlled HTTP, database, static-analysis, or package-metadata services expose deterministic resources with tunable latency. This demonstrates that the mechanism is not filesystem-specific and allows direct study of the wall-clock value of avoiding repeated external calls.

12.10 Family J: Naturalistic coding-agent tasks

After controlled evaluation, instrument selected Paper 4.5 tasks, SWE-bench Verified subsets, Mini-SWE-agent subsets, and custom multi-issue repository tasks. First measure how frequently independent subagents naturally repeat reads/searches already performed by ancestors. Then evaluate end-to-end benefit without forcing reuse through scripted behavior.

13 Baselines

The minimum comparison set is:

1. conventional isolated subagents;
2. conventional subagents with harness-level result memoization;
3. PRA records without cross-agent visibility;
4. PRA ancestor visibility with payload reuse;
5. PRA ancestor visibility with native K/V reuse;
6. PRA with resource-level selective invalidation.

Where practical, also compare parent-context copying/snapshotting and full transcript sharing.

Separating harness memoization from K/V reuse is essential to avoid attributing ordinary tool caching gains to PRA.

14 Hypotheses

H1: Result reuse. Repeated ancestor reads/searches reduce tool executions and wall-clock latency when children can reuse valid records.

H2: Native K/V reuse. For sufficiently large shared records, native K/V reuse reduces child prefill cost beyond harness-level memoization.

H3: Bounded context. PRA preserves bounded active physical context while avoiding repeated copying/prefill of shared material into every child.

H4: Selective invalidation. Resource-level invalidation preserves more reusable state than domain-wide invalidation while maintaining zero stale-read errors under stated consistency assumptions.

H5: Descendant permeability. Parents answer post-hoc evidence questions more accurately and with lower context growth by routing into completed descendants than by relying on summaries alone.

H6: Fan-out scaling. Benefits increase with shared context size, child fan-out, and repeated-access probability, subject to routing and cache-residency overhead.

15 Analytical Cost Model

Let S be the token size of a reusable shared record set, N the number of child agents, and p the probability that a child needs the shared set. Let C_{tool} be the external tool cost, $C_{prefill}(S)$ the model prefill cost, C_{route} the PRA lookup/routing cost, and C_{mat} the cost of materializing already stored K/V.

Ignoring one-time parent acquisition, duplicated work without reuse is approximately

$$C_{base} = Np\left(C_{tool} + C_{prefill}(S)\right).$$

With cross-agent PRA reuse,

$$C_{pra} = Np\left(C_{route} + C_{mat}\right).$$

The experiments should report empirical break-even regions rather than assuming universal benefit.

16 Metrics

16.1 Correctness

Task success, patch/test success, stale-read count, invalid reuse count, and post-hoc evidence accuracy.

16.2 Tool and agent behavior

Tool calls per agent, duplicate calls, avoided calls, reused reads/searches, invalidations, false invalidations, and explicit cache hit/miss reasons.

16.3 Model cost

Logical tokens, physically materialized tokens, prefill tokens, decode tokens, reused K/V token-equivalents, and K/V bytes where measurable.

16.4 Runtime

Wall-clock time, external-tool latency, routing overhead, materialization overhead, time-to-first-child-result, and total run time.

16.5 Storage

Duplicate payload bytes avoided, K/V storage overhead, cache residency, and evictions.

17 Tracing and Scientific Instrumentation

Every reuse attempt should emit a structured trace record containing request/source agent identities, source record, resource identity, decision, reason, validation method, age, causal position, and whether payload, K/V, or tool execution was reused.

Suggested miss reasons include:

```
NO_MATCH
NOT_VISIBLE
EXPIRED
WRITE_INVALIDATION
EXTERNAL_VALIDATION_FAILED
MODEL_INCOMPATIBLE
KV_EVICTED
UNKNOWN_EFFECT
POLICY_DISABLED
```

This instrumentation is required both for systems debugging and for credible experimental analysis.

18 Implementation Status

Table 2 separates implemented runtime contracts from mechanisms that still need live engine or natural-agent validation. The native interface is a narrow callback: an engine returns a receipt containing model, revision, encoding, position contract, token count, and byte count. A reuse request must match all compatibility fields, and the harness attaches a given receipt at most once per target agent.

Table 2: Paper 9 implementation boundary. "Experimental" denotes implemented code without a production or natural-agent qualification claim.

Mechanism	Status	Current evidence
Agent identity and start/stop/resume records	Supported	Persistence and graph-replay tests
Ancestor selected/routable visibility	Experimental	Runtime contract tests
Completed-descendant visibility	Experimental	Runtime contract tests
Exact tool/payload reuse	Experimental	Runtime tests and SCRB
Resource-level and external validation	Experimental	Mutation and validator tests
Native K/V compatibility callback	Experimental	Receipt and exactly-once tests
Parallel scheduler and sibling communication	Harness-owned / open	No end-to-end cohort
General DAG joins	Not implemented	Deferred
Natural subagent campaign	Open	Single-agent opportunity audit only

19 Non-Goals

The first Paper 9 implementation does not attempt cross-session long-term memory, arbitrary distributed cache coherence, automatic semantic equivalence between different tool calls, database transaction semantics, general peer-to-peer multi-agent communication, or a new delegation/planning algorithm. These are separable research directions.

20 Controlled Results

20.1 Protocol and interpretation boundary

SCRB v1 contains 1,600 scaling rows: five workload seeds crossed with four shared-record sizes (2K, 8K, 32K, and 64K tokens), four child fan-outs (2, 4, 8, and 16), four reuse probabilities, and five execution conditions. It is an event-driven simulation whose declared constants are 50 ms per tool execution, 0.020 ms per prefill token, 0.120 ms per PRA lookup, and 0.0015 ms per native token materialized. These values expose the analytical break-even surface; they are not observations from a model server. Separate tests execute the actual lineage, visibility, invalidation, external-validation, graph-replay, and native-receipt code paths.

20.2 Tool reuse is not native reuse

Table 3 reports the fully shared 8K/eight-child point. Ordinary memoization and PRA payload reuse each remove eight external tool executions, reducing the modeled session cost by about 21%, but both still prefill the shared result for every child. Native reuse removes the same tool executions and reuses 65,536 token-equivalents of K/V, reducing repeated prefill by 88.9%. At the declared 4,096 native bytes per token, this also avoids 256 MiB of duplicate physical K/V allocation across the eight child streams. This byte value is a cost-model parameter, not measured residency.

Table 3: SCRB controlled point: 8K shared tokens, eight children, reuse probability one. Cost is simulated from the declared constants.

Condition	Tool calls	Prefill tokens	Cost (ms)	Speedup
Isolated contexts	9	73,728	1,924.6	1.00×
Harness memoization	1	73,728	1,524.8	1.26×
PRA, no visibility	9	73,728	1,925.5	1.00×
PRA payload reuse	1	73,728	1,525.5	1.26×
PRA native K/V reuse	1	8,192	313.1	6.15×

Figure 1 shows the full-fan-out slices. The native condition becomes more favorable as fan-out grows because the parent's initial acquisition and encoding are amortized. The exact magnitude depends on engine bandwidth, layer count, cache placement, and record size; only the qualitative scaling relation is supported here.

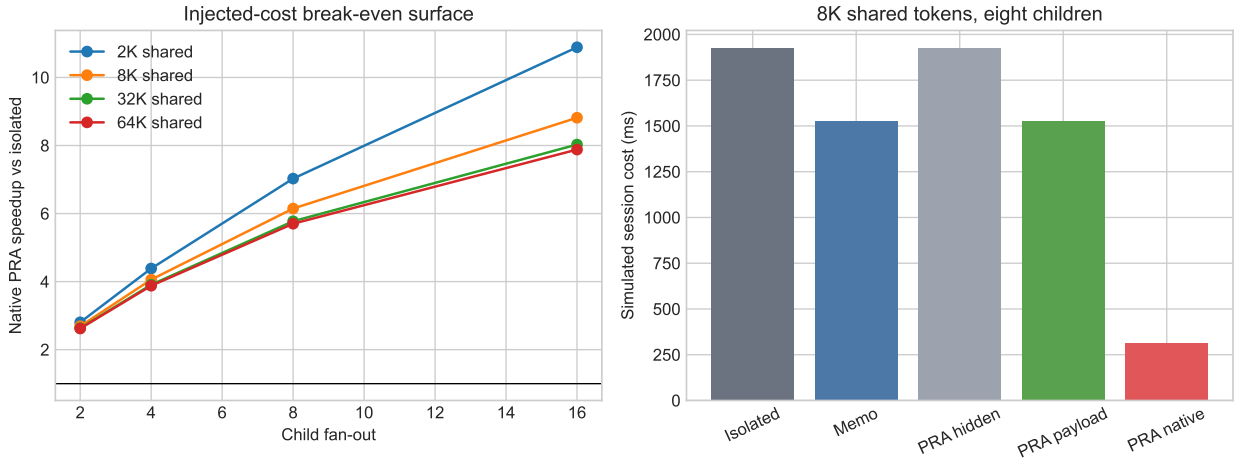

Figure 1: Controlled SCRB scaling. Left: native-reuse speedup under the injected cost model. Right: decomposition at the 8K/eight-child point. "PRA hidden" is the no-cross-stream-visibility control.

20.3 Selective invalidation and descendant evidence

The mutation fixture gives every child one read of modified resource A and one read of unchanged resource B. At fan-out 16, domain-wide invalidation executes all 32 reads and causes 16 false invalidations per run. Resource-level invalidation reuses all 16 reads of B, re-executes all reads of A, and produces no stale read. The deliberately unsafe control reuses all 32 and produces 16 stale reads. The production runtime check independently confirms the same decision sequence: two executions for A, one for B, and explicit `WRITE_INVALIDATION` for the rejected reuse.

The completed-child fixture asks a parent for exact detail omitted from roughly half of compact summaries. Summary-only return reaches 55.2% over 500 seeded cases. Full transcript copy and replay reach 100% while adding 16,384 tokens to parent context. Exact descendant routing also reaches 100% while materializing 384 tokens, a 42.7× reduction in context growth relative to the full-transcript controls. This is an oracle-addressed controlled fixture; learned descendant selection remains unmeasured.

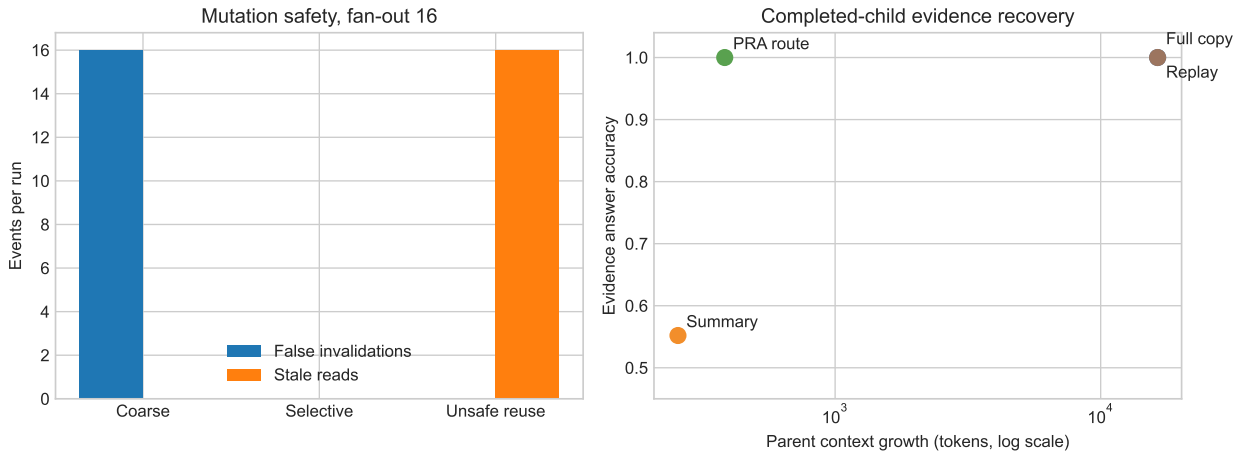

Figure 2: Left: selective invalidation preserves valid reuse without the stale reads of the unsafe control. Right: controlled completed-child evidence accuracy versus parent-context growth.

20.4 Natural trace opportunity audit

We scanned 12 existing Paper 4.5 coding-agent trajectories containing 90 tool calls. Forty were classified as read/search operations; eight of those were exact repeats, a 20% operation-level rate, concentrated in two traces. This is not a subagent experiment: the source trajectories contain one agent each. It only establishes that repeated acquisition occurs in a small natural coding sample and justifies instrumenting a real delegated cohort. It cannot estimate ancestor hit rate, native reuse, or end-task benefit.

21 Related Work

ReAct interleaves reasoning and external actions, while SWE-agent emphasizes the agent-computer interface for repository work [9, 7]. OpenHands provides a general software-agent platform with sandboxed execution and multi-agent coordination [5]. AutoGen makes multi-agent conversation and configurable agent interaction first-class application primitives [6]. These systems motivate the conventional spawn/run/return harness used here; Paper 9 does not propose a new delegation planner.

Prompt and prefix-cache systems reuse computation attached to shared token prefixes [1, 8, 2]. Harness memoization reuses an operation result, while semantic caches may match similar requests. Paper 9 separates both from two additional mechanisms: a typed result can remain addressable without being copied into every stream, and compatible native K/V can be referenced across non-prefix agent branches. Resource identities and invalidation epochs resemble dependency tracking in storage and build systems, but the first implementation intentionally stops short of a general transaction or distributed-coherence protocol.

22 Limitations and Open Gates

The mechanism may provide large gains only when agents repeatedly need shared records or expensive tools. Controlled orchestration can overstate natural reuse; the 12-trace audit is too small and is not multi-agent. The injected latency model omits queueing, transfer, cache eviction, and

engine scheduling. Likewise, the native callback validates compatibility and exactly-once attachment semantics but does not yet measure model outputs or live prefill.

Correctness depends on declared consistency assumptions. External mutation can invalidate local state even when no write appears in the session tree. The runtime therefore rejects `UNKNOWN` effects and requires a validator for `EXTERNAL` resources. The next gates are a real sequential/parallel subagent cohort, a live compatible engine comparison against harness memoization, and learned rather than oracle descendant routing. Cross-session memory remains explicitly outside Paper 9.

The present result is therefore architectural and controlled: subagent isolation and context sharing need not be binary, and the implementation can make boundaries selectively permeable without weakening its default safety policy. Whether that mechanism is economically useful in natural workloads is not yet established.

23 Conclusion

Paper 9 extends PRA from task-aware context within one agent stream to multiple subagent streams with explicit lineage and selectively permeable boundaries. The harness remains conventional and configurable; concrete tool semantics stay in harness adapters; the PRA runtime remains generic and potentially remote; and the model engine handles native attention state. The implementation shows that exact ancestor reuse, selective invalidation, durable graph replay, and completed-descendant routing can share one coherent typed-record contract. Controlled SCRB results expose the distinct value of tool, payload, and native reuse and show the expected fan-out scaling without presenting simulated costs as live serving evidence. The remaining empirical question is deliberately narrower: how often natural subagents encounter valid reuse opportunities, and whether live native K/V reuse pays after routing, transfer, residency, and scheduling costs are included.

References

[1] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *arXiv preprint arXiv:2311.04934*, 2023.

[2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. Preprint: `https://arxiv.org/abs/2309.06180`.

[3] Jorge Simao. Progressive retrieval attention: Bounded sparse native-KV for addressable logical memory. Companion Paper 1 manuscript, 2026.

[4] Jorge Simao. Progressive retrieval attention for pretrained transformers: Sparse native-K/V memory with semantic routing. Companion Paper 2 manuscript, 2026.

[5] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

[6] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[7] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[8] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *arXiv preprint arXiv:2405.16444*, 2024.

[9] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.